COMPUTER ENGINEERING
QUEZON CITY

Final Project: Student Attendance Monitoring System

Course Code: CPE 010 **Course Title: Data Structures and Algorithms**

PROJECT INFORMATION

Item	Details
Project Title	Student Attendance Monitoring System
Business Process Emulated	Academic Attendance Tracking and Student Record Management Process
Section / Group No.	CPE21S1 / Group 1
Instructor	Engr. Jimlord Quejado
Date Submitted	November 6, 2026
GitHub Repository	https://github.com/Medinacpe/Group-1-CPE21S1-Attendance-Monitoring-Final-Project (details in Appendix C)

Group members, roles, and contributions are listed in **Appendix B. Team Composition**.

Intended Learning Outcomes Addressed

At the end of this activity, the students should be able to:

- Apply fundamental data structures and techniques to solve engineering-related problems through algorithm design.
- Design efficient algorithms and analyze their complexity.
- Build efficient algorithms based on selected data structures using good programming styles in C++.

Objectives

General Objective: To develop a successful C++ program that emulates a student attendance monitoring program using appropriate data structures and algorithms. Moreover, a few additional features added such as automatic grouping, searching, and fundamental quality of life elements are implemented within the project, carrying a total of 5 different main algorithms.

Specific Objectives:

1. To manage student attendance using a queue so that teachers could check the attendance by a first-come, first-served basis alphabetically.
2. To search student names and attendance records efficiently using binary search.
3. To sort student names and attendance records using merge sort.
4. To analyze the time complexity of the main operations theoretically and experimentally.

1. BUSINESS PROCESS

1.1 Organization and Business Process

The organization that is considered in this project is a realistic school where teachers have to manually record student attendance using paper, attendance sheets, or even spreadsheets. The process involves preparing a class list, checking the presence of each student, as well as marking if they are present, late, or absent. Additionally, inputs include that of student names, the class session's data and time, as well as attendance statuses. The output shall be a completed record of attendance that teachers may use to record, view, and monitor student participation.

Inspired by Canvas' attendance monitoring system, the proposed Student Attendance Monitoring System shall emulate the process via C++ in a console-based program that shall allow teachers to add or import student names, check their attendance, update records, search for students, view the attendance history, as well as generate student groupings. Student information and attendance records are saved in a master CSV file for future use and the program is meant to be implemented for a single class.

1.2 Current Process (As-Is)

Step	Activity	Performed By	Data Involved
1	Teachers adds student names to the list	Teacher	Student names
2	Record date and time of the class session	Teacher	Session date and time
3	Attendance checking; calling each student's names and seeing if they're in class.	Teacher	Student names, student presence
4	Mark each student as present, late, or absent.	Teacher	Attendance status
5	Review attendance entries and rectify errors.	Teacher	Student names, Attendance status
6	Update class list when students are added or removed.	Teacher	Student Info.
7	See and review previous attendance sheets when student attendance history is needed.	Teacher	Previous attendance records
8	Store completed attendance sheet, saving for future reference.	Teacher	Complete attendance records

1.3 Problems in the Current Process

1. Manually inputting student names can be time consuming for the user, especially when the classes handled are large. Additionally, this can also lead to errors like names that are misspelled or even duplicate entries.
2. When searching for the attendance of specific students, it may become difficult to handle along the way as the numbers of students increase, even more so if the records are not properly organized or sorted.
3. Dealing with the management and modification of attendance records can lead and are prone to human errors, like with marking the wrong status of attendance of a student or the accidental changing of a student's already correct information.

1.4 Proposed System

#	Feature	Business Task It Supports
1	Attendance Checking	Process students alphabetically as well as recording each student as either present, late, or absent for a specified date and time.
2	Attendance History & Correction	Allows teachers to view and review students' previous attendance records and rectify errors.
3	Student Grouping	Create random or alphabetical round robin groups with an equal number of students (if applicable).
4	Student Searching	Finds a student using their full name as well as displaying their attendance history.
5	Student Record Management	Allows teachers to be able to add, import, display, rename, as well as delete students with changes being saved to the main CSV file.

1.5 Scope and Delimitations

SCOPE

- This system covers the recording of student attendance, creation of randomized or patterned sets of groupings, as well as looking up any individual student and keeping track of attendance history as per date and time. The main target user for the program includes teachers that shall monitor and manage the attendance of students and the tracking of their attendance in class. Furthermore, student information as well as attendance records are stored locally within a master CSV file and are loaded when the program starts.

DELIMITATIONS

- The system only supports one class as well as one user at a time and does not provide separate accounts or or managing multiple classes and/or subjects. Additionally, the program uses a console interface and local files, not including any GUI, database, and etc. Attendance is entered by the teacher and the system does not automatically detect students through biometrics, QR codes, or other devices. Moreover, student searching requires the full name where capitalization does not affect the search at all and names that are

only partial in form are not supported. Generated groupings shall only include all students currently in class and are displayed without being saved. Lastly, the system does not handle any grades, assignments, activity submissions, or any other school service that wouldn't involve attendance checking.

2. DATA STRUCTURES AND ALGORITHMS APPLIED

2.1 Summary of Data Structures and Algorithms Applied

#	Data Structure / Algorithm	Where It Is Used in the Business Process	Main Operations
1	Doubly Linked List	Store students and attendance records in memory as well as supporting the adding and removing of records.	addStudent, addRecord, removeRecord, manageStudents
2	Queue using a Linked List	Processing of students one at a time during attendance checking, following alphabetical enqueue order.	enqueue, dequeue, peek, IsEmpty
3	Binary Search	Find a student by their full name in a sorted array of student pointers.	binarySearch
4	Merge Sort	Sorting student pointers alphabetically for displaying the students, checking attendance, searching, as well as grouping.	mergeSort, merge, sortedStudents
5	Linear Search	Finding specific students in the doubly linked list when viewing, editing, or deleting student records.	findID, findName, findRecord

2.2 Linked List (Doubly)

Concept: A doubly linked list stores data and nodes that is connected to the previous and the next node

Application in the System: The program implements separate doubly linked lists for student and attendance records where student nodes store both an ID as well as a name, while attendance nodes store the student ID, session date, session time, as well as the attendance status. When new nodes are added, they are put at the end of their respective lists and when nodes are deleted, their student attendance records are also deleted accordingly.

Justification: Linked lists allow stored data to grow without a fixed capacity within an array. The nodes implemented can simply be added or removed by updating pointers without shifting the other elements.

Operations and Complexity: Appending nodes using the tail pointer: O(1), Removing a known attendance node:O(1), Searching or traversing student list: O(n), Searching or traversing attendance list: O(r) where n is number of students and r is the number of attendance records, Deleting a student and associated records: O(n + r).

2.3 Queue Linked List

Concept: A queue using a linked list is a dynamic implementation of the First-In, First-Out (FIFO) queue data structure. Instead of using a fixed-size array, it uses a chain of dynamically allocated memory blocks called nodes.

Application in the System: The program first sorts the students alphabetically followed by adding students without any existing entry for a specific section and are added to the queue in said order. The teacher then marks the student at the front as either present, late, or absent where the program removes that queue node and proceeds to the next student after.

Justification: By implementing the queue, it provides a clear order for checking the attendance by using the rear and front pointers for allowing students to be added at the back and removed from the front without shifting the other entries.

Operations and Complexity: enqueue, dequeue, peak, and IsEmpty: O(1). Processing all n queued students: O(n).

2.4 Binary Search

Concept: A binary search is an efficient algorithm that finds a target value within a sorted array by repeatedly dividing the search space in half.

Application in the System: The program creates an array of student pointers and sorts that alphabetically. Binary search then compares the entered full name with the names in said array, ignoring any capitalization from said names. Then, once a match is found, the program displays the student's details and separately scans the attendance list to show the history.

Justification: Using an array allows for direct access to the middle element, making binary search very suitable for the sorted student array. Once the array is prepared, binary search generally requires fewer comparisons than checking all of the students individually like with linear search.

Operations and Complexity: Search: O(log n), Best case: O(1), Worst case: O(log n)

2.5 Merge Sort

Concept: This algorithm works by breaking arrays into individual parts and sorting those pieces together in order

Application in the System: Sorts the student records alphabetically by their name before starting attendance checking or running a binary search. The program takes student pointer references from the doubly linked list and puts them in a continuous array. It splits array of the student names in half until the single items remain, it then compares the names alphabetically and merges them back into a single sorted list. Lastly, it feeds this sorted list into queue for attendance marking

Justification: Merge sort always runs in O(N log N) time, making it faster than bubble sort or insertion sort. Unlike quicksort, it does not slow down to O(N^2) on bad data, giving it consistent performance regardless of how the list starts..

Operations and Complexity: The system uses the mergeSort and merge functions. It runs in O(N log N) time complexity for best, average and worst cases, and uses O(N) extra space to merge temporary arrays.

2.6 Linear Search

Concept: Linear search checks a list one item at a time from the start until it finds a match or reaches the end. It works even if the data is not sorted.

Application in the System: The function, '**findID**' searches the student list for a matching ID with '**findName**' checking for a matching name (including the checking during names that are duplicate). Additionally, '**findRecord**' searches attendance records for a matching student ID, date, as well as time. The searches support student management, attendance checking, as well as the recording of corrections.

Justification: Binary search needs sorted data, and a linked list cannot jump to the middle. Linear search works on the list as it is and still works right after new students are added.

Operations and Complexity: Looking students up best case: O(1), worst case O(n). Finding attendance entry best case: O(1), worst case O(r).

3. SYSTEM FLOWCHART

3.1 Main System Flowchart

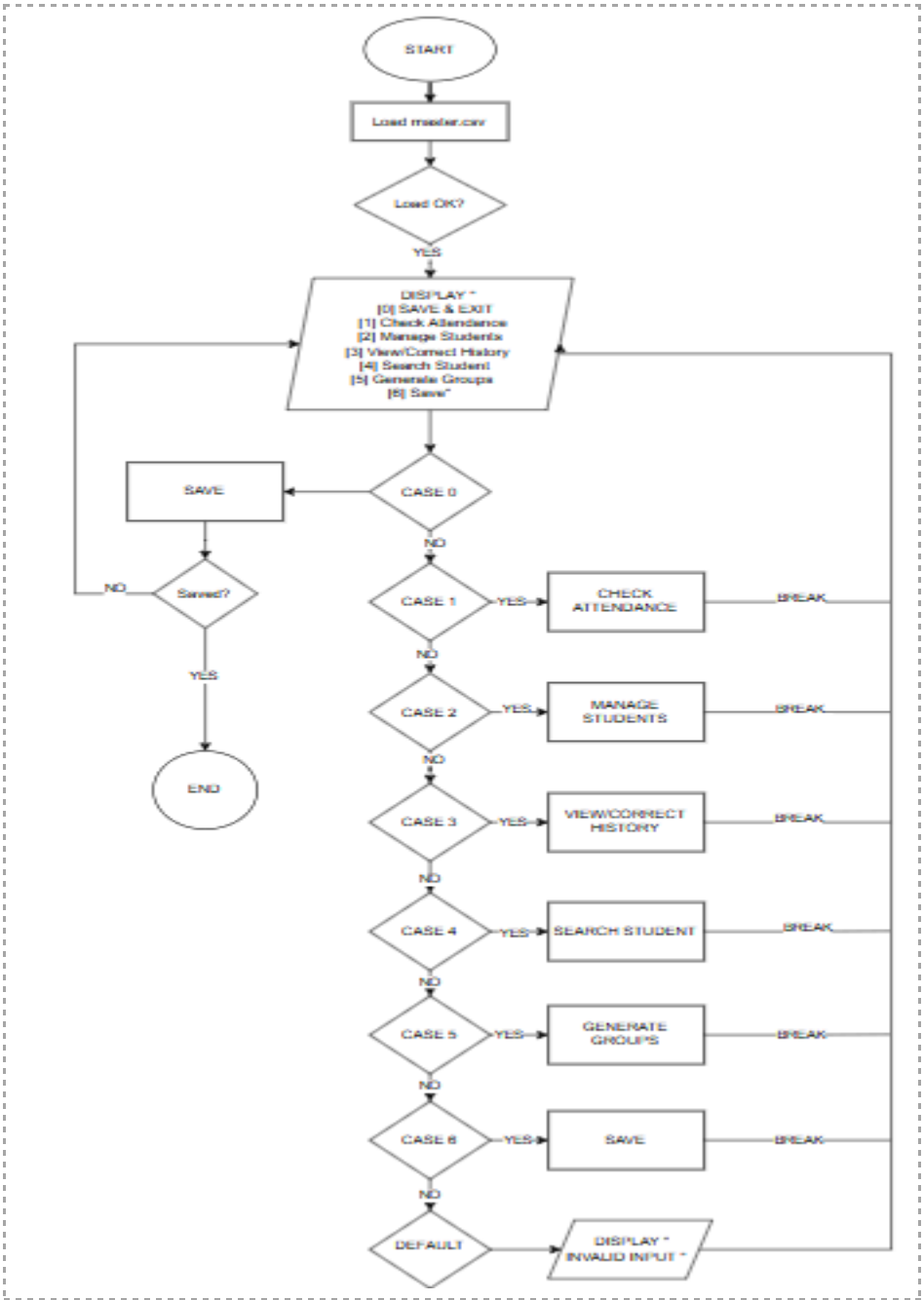


Figure 1a. Main Flowchart of Student Attendance Monitoring System

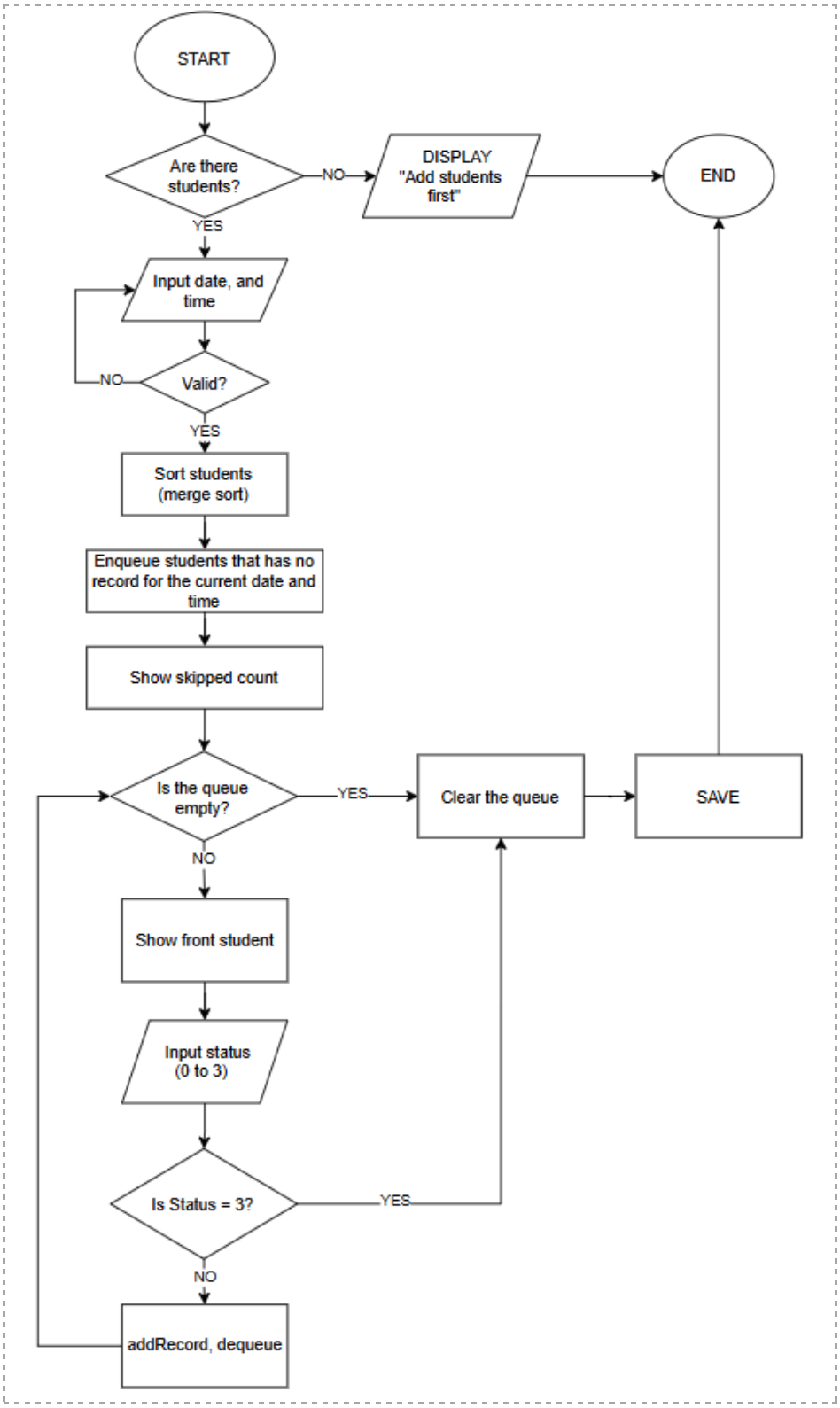


Figure 1b. Check Attendance Function Flowchart

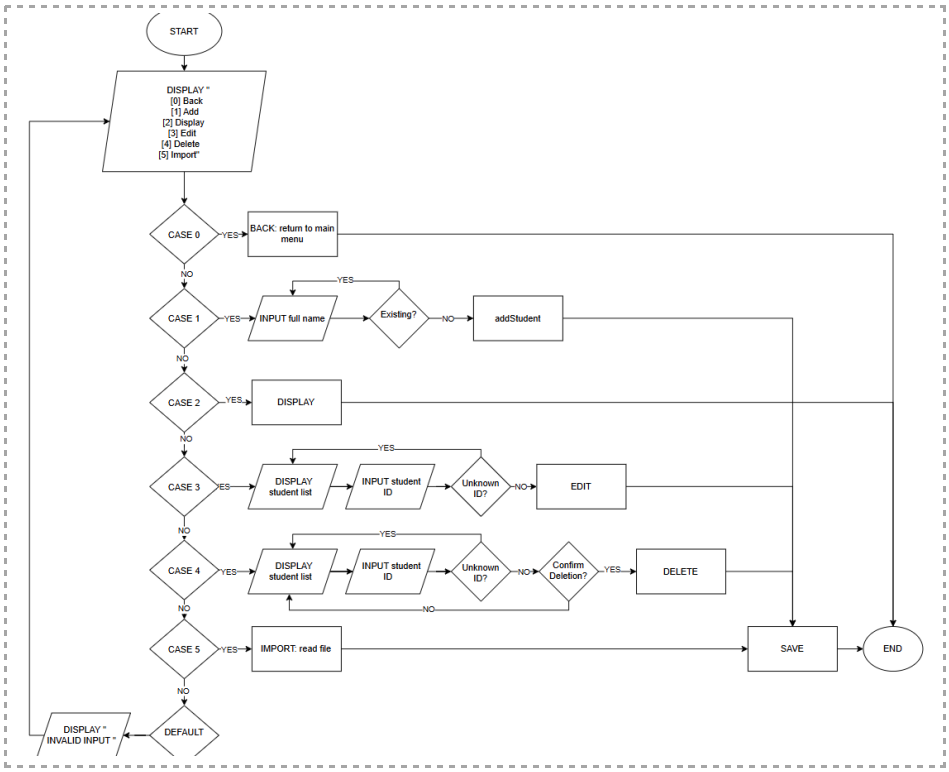


Figure 1c. Manage students Function Flowchart

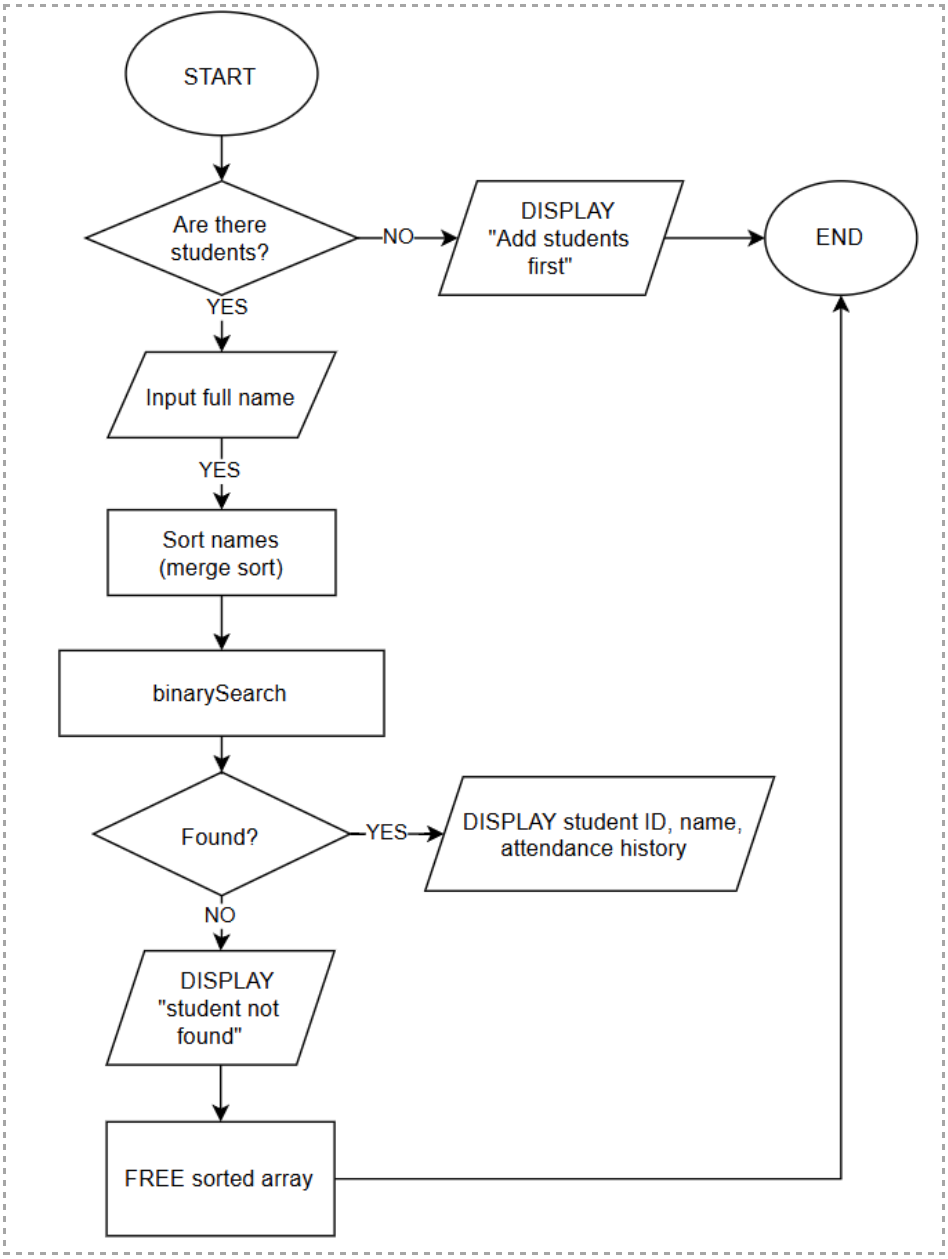


Figure 1d. Search Student Function Flowchart

3.2 Flow Description

1. The program starts and loads the saved student info and attendance records from the master csv file. If the file cannot load properly, then the program will display an error message and will stop without overwriting the current data.
2. Once the data has been loaded, the main menu displays 7 choices; [0] Save and Exit, [1] Check Attendance, [2] Manage Students, [3] View/Correct History, [4] Search Student, [5] Generate Groups and [6] Save.
3. If the user chooses 1, the system will proceed to execute the attendance checking function, and loops back to the main menu after the function ends.

4. If the user chooses 2, the system will then proceed to execute the student management function, and loops back to the main menu after the function ends.
5. If the user chooses 3, the system will then proceed to display the student attendance history and allows the user to correct any existing attendance entries, and loops back to the main menu after the function ends.
6. If the user chooses 4, the system will search for a student using their full name after the teacher inputs it through a binary search algorithm and also display their information and also their attendance history if found, then loops back to the main menu after the function ends.
7. If the user chooses 5, the system will generate a student group using either alphabetical round robin or just random grouping, then returns to the main menu once the groups are displayed.
8. If the user chooses 6, the system will save the current student info and the attendance record to the master csv file then return to the main menu.
9. If the user chooses 0 then the system will save all data before exiting the program. Then if the saving fails to do so, the user can still retry and return to the main menu.
10. If the user enters an invalid choice, then the system will display an invalid input message then goes back to the main menu.

4. OUTPUT

Feature	Figure No.	Input Used
Attendance Checking	Figure 2	Actual date, student names, and attendance statuses entered
Attendance History & Correction	Figure 3 (a-b)	Student name, attendance date, and corrected status
Student Grouping	Figure 4	Grouping method and number of groups/group size requested by the program
Student Searching	Figure 5 (a-b)	Exact student name entered
Student Record Management	Figure 6 (a-c)	Test student's original name, updated name, and deletion choice
Invalid Input Handling	Figure 7 (a-b)	Menu number outside of the scope.

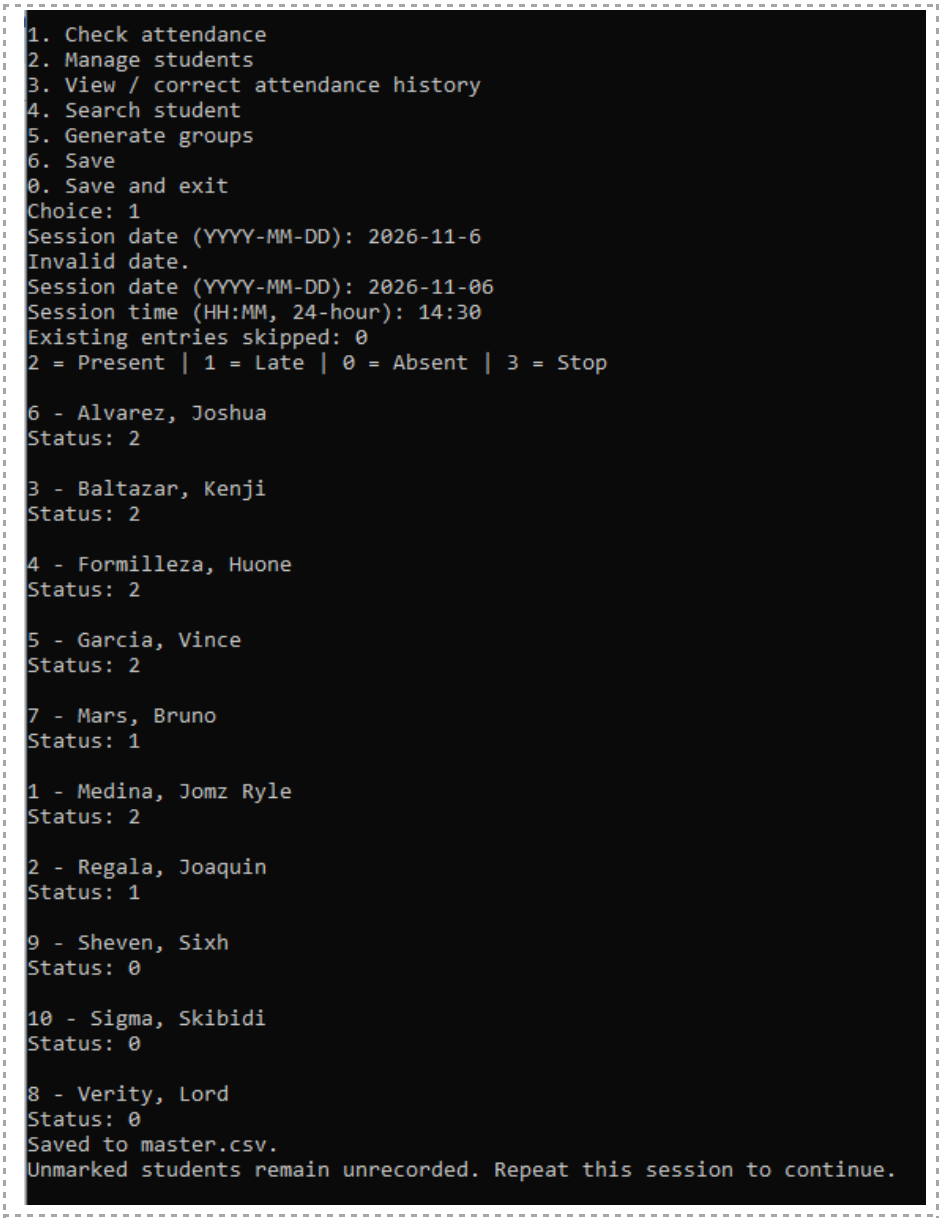


Figure 2. Attendance Checking - Processing attendance of pre-imported students and saving of inputs.

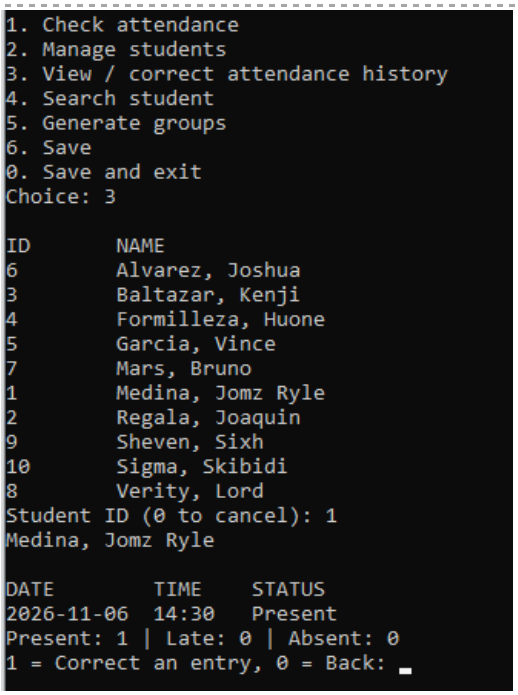


Figure 3-a. Attendance History - Displaying a student's attendance history.

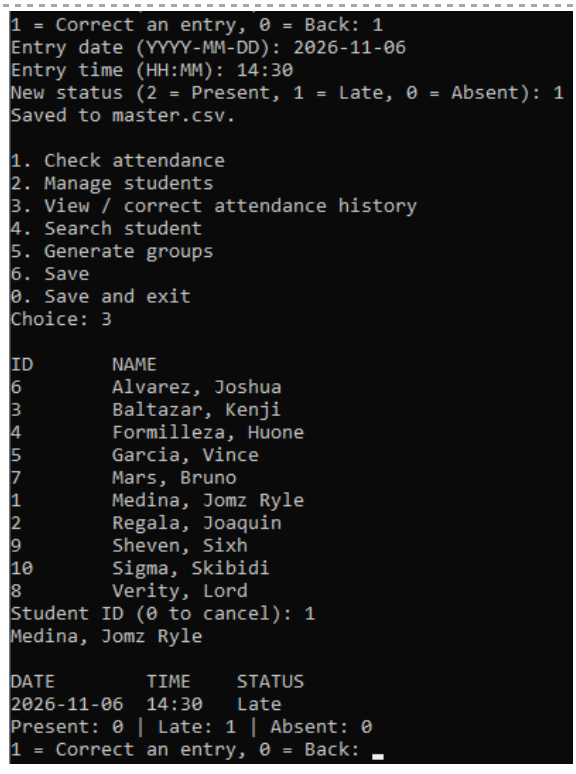


Figure 3-b. Attendance Correction - Changing attendance status followed by verification of its change.

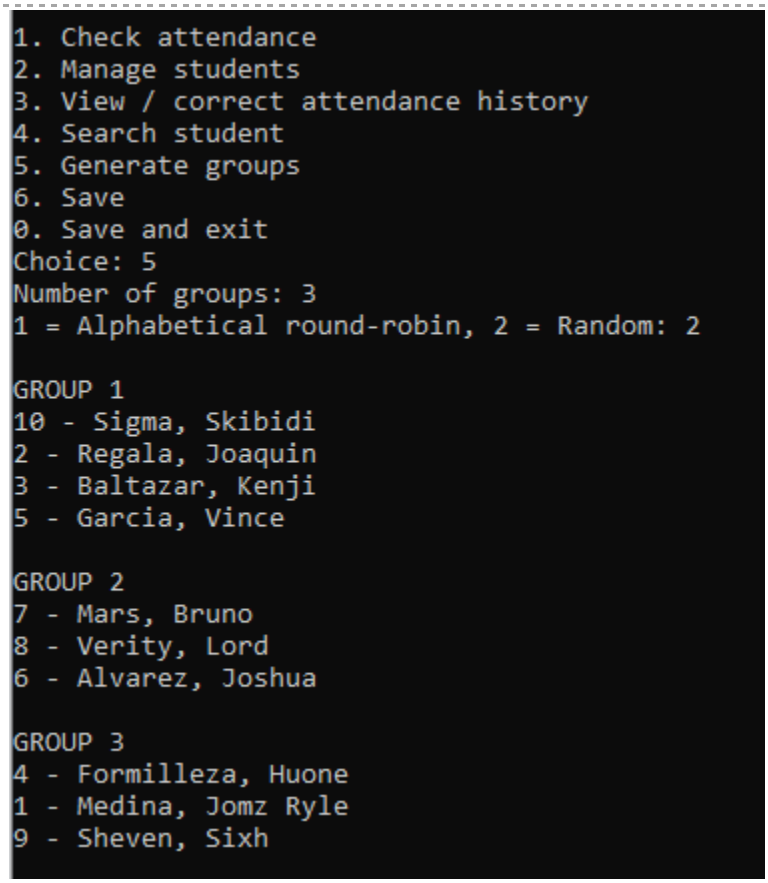


Figure 4. Student Grouping - Students grouped via randomized setup.

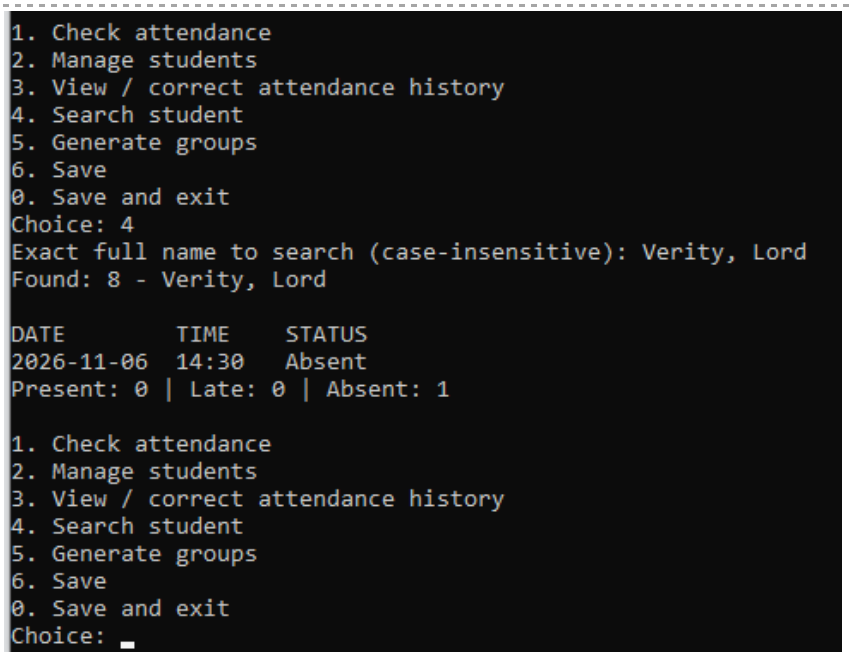


Figure 5-a. Student Searching - Finding a student and showing their attendance history.

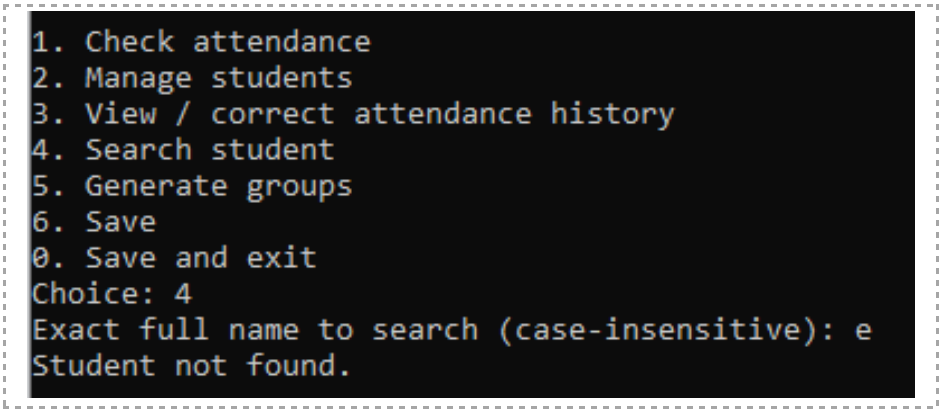


Figure 5-b. Student Searching - Finding an invalid student (not in list).

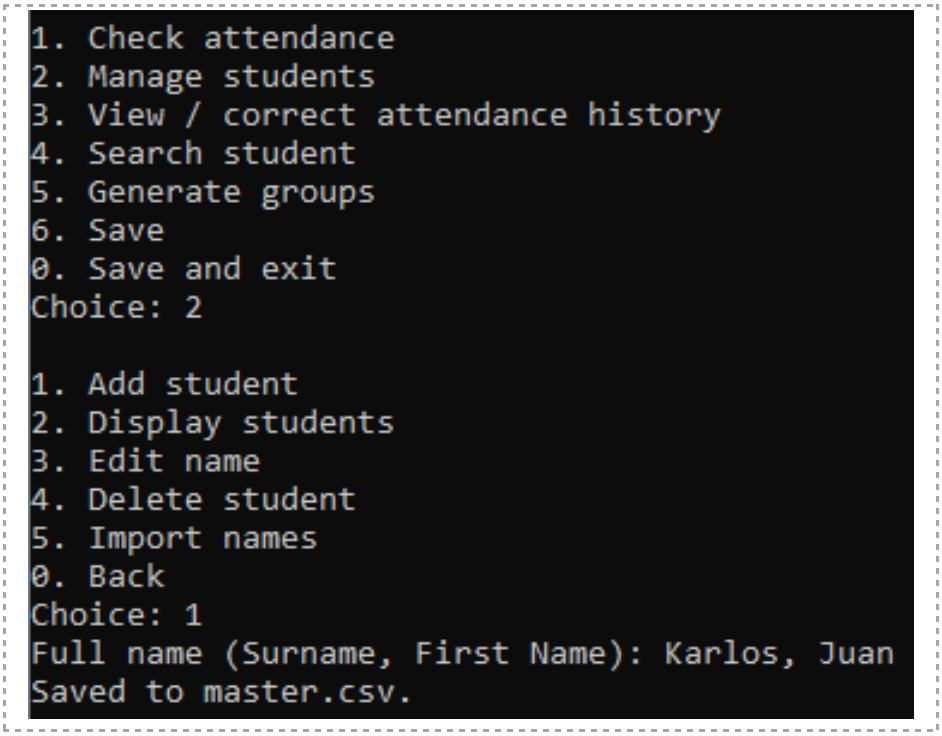


Figure 6-a. Student Record Management - Adding a student.

```
1. Check attendance
2. Manage students
3. View / correct attendance history
4. Search student
5. Generate groups
6. Save
0. Save and exit
Choice: 2

1. Add student
2. Display students
3. Edit name
4. Delete student
5. Import names
0. Back
Choice: 3

ID       NAME
6        Alvarez, Joshua
3        Baltazar, Kenji
4        Formilleza, Huone
5        Garcia, Vince
11       Karlos, Juan
7        Mars, Bruno
1        Medina, Jomz Ryle
2        Regala, Joaquin
9        Sheven, Sixh
10       Sigma, Skibidi
8        Verity, Lord
Student ID (0 to cancel): 11
New full name: Farmer, Aura
Saved to master.csv.
```

Figure 6-b. Student Record Management - Editing a student's name.

```
1. Check attendance
2. Manage students
3. View / correct attendance history
4. Search student
5. Generate groups
6. Save
0. Save and exit
Choice: 2

1. Add student
2. Display students
3. Edit name
4. Delete student
5. Import names
0. Back
Choice: 4

ID       NAME
6        Alvarez, Joshua
3        Baltazar, Kenji
11       Farmer, Aura
4        Formilleza, Huone
5        Garcia, Vince
7        Mars, Bruno
1        Medina, Jomz Ryle
2        Regala, Joaquin
9        Sheven, Sixh
10       Sigma, Skibidi
8        Verity, Lord
Student ID (0 to cancel): 11
Delete Farmer, Aura and ALL their attendance history?
1 = Yes, 0 = Cancel: 1
Saved to master.csv.
```

Figure 6-c. Student Record Management - Deleting the test student.

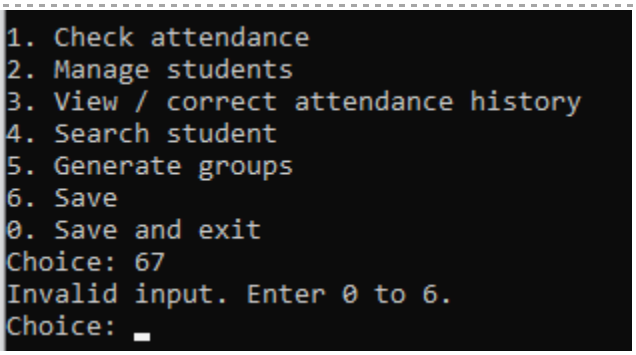


Figure 6-a. Invalid Input Handling - Entering a number that is out of the range of the menu choices.

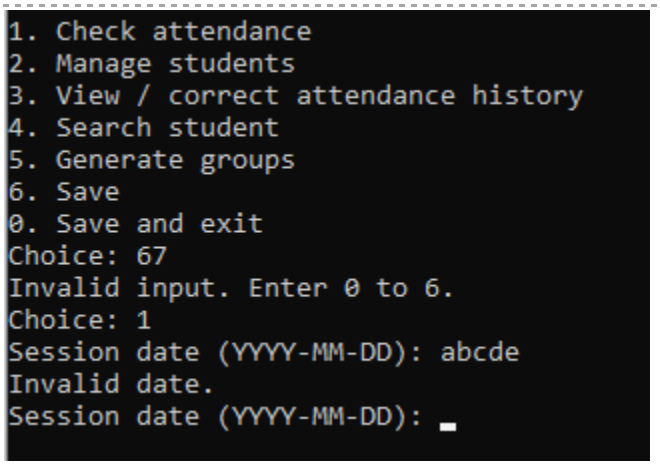


Figure 6-b. Invalid Input Handling - Entering letters when asked for a date.

5. PERFORMANCE ANALYSIS

5.1 Theoretical Analysis

This section discusses the time and space complexities of the main operations in the Student Attendance Monitoring System. The time complexity of each operation is given below in Big O notation. We have provided the best case, average case and worst-case time complexity for each operation. N is the number of students in the class. Space complexity refers to the amount of additional memory required by the operation. The table below summarises the time and space complexities and the derivations are discussed below.

Operation	Data Structure / Algorithm	Best	Average	Worst	Space
Enqueue or dequeue one student	Linked-List Queue	O(1)	O(1)	O(1)	O(N)
Sort the student pointer array	Merge Sort	O(n log n)	O(n log n)	O(n log n)	O(N)
Search an already sorted student array	Binary Search	O(1)	O(log n)	O(log n)	O(1)

Find a student by ID or name	Linear Search	O(1)	O(n)	O(n)	O(1)
Find an attendance entry	Linear Search	O(1)	O(r)	O(r)	O(1)
Append one student node using the tail pointer	Doubly linked list	O(1)	O(1)	O(1)	O(1)
Remove a known attendance node	Doubly linked list	O(1)	O(1)	O(1)	O(1)
Display one student's complete attendance history	Attendance-list traversal	O(r)	O(r)	O(r)	O(1)

Derivation:

1. **Attendance Checking - Linked List Queue**

   This uses a linked list queue. This means that it uses the FIFO method. As we are only updating the front and back pointers, we can enqueue and dequeue in O(1). We have an O(1) time complexity. However we have an O(N) space complexity as we need to store student records.

2. **Sorting Student List - Merge Sort**

   Merge Sort will split the list into halves until there is only one item per list. Then compare the items and merge them. This will have a time complexity of O(N log N) in the best, average and worst case. This is because we are splitting the list approx log N times. We then have N items to process at each level. We also have an O(N) space complexity as we need extra memory to store the items.

3. **Student Lookup - Binary Search**

   Binary Search checks the midpoint of a sorted array of student pointers. If the target matches immediately, the best-case complexity is O(1). If not, it eliminates half of the remaining items each step (N, N/2, N/4, ...). Halving the search space recursively yields $\log_2 N$ steps, making both average and worst-case time complexity O(log N).

4. **Direct Record Search & Modification - Linear Search**

   Linear Search traverses the doubly linked list sequentially starting from the head node. If the target record is the first element, best-case time complexity is O(1). If the target is at the end or missing entirely, it checks all N nodes, resulting in a worst-case time complexity of O(N). On average, checking N/2 nodes simplifies asymptotically to O(N).

5. **Attendance History**

   Displaying a student's complete attendance history needs checking all of the attendance records in order to see all matching entries. Because of that, traversal takes O(r) time even when the student has no attendance records where the traversal pointer and attendance counters also require O(1) auxiliary space.

5.2 Experimental Analysis

Setup Item	Details
Computer (CPU, RAM)	Intel Core i7-7700, 32 GB RAM
Operating system	Windows 10
Compiler and settings	GNU g++ 16.1.0, C++11, -O2 optimization
Timing method	Std::chrono::steady_clock, measuring only the operation that is being tested (data preparation, console output, as well as file writing are excluded from the timing sections).
Input data	Includes automatically generated student records with input sizes 100, 1,000, 10,000, and 100,000 where linear and binary search use the same nonexistent target name. Additionally, binary search uses a sorted array and merge sort uses an input that is alphabetically but reversed.
Repetitions	5 runs per input size. Each run does 100 searches with the time divided by 100 in order to obtain the average in each search. Additionally, merge sort runs once per run and results are averaged across the five runs.

Input Size (n)	Linear Search Time (ms)	Binary Search Time (ms)	Merge Sort Time (ms)
100	0.0082336	0.0002454	0.01254
1,000	0.1029548	0.0005564	0.2766
10,000	1.0877778	0.0005324	3.42186
100,000	12.9251674	0.0006898	50.82048

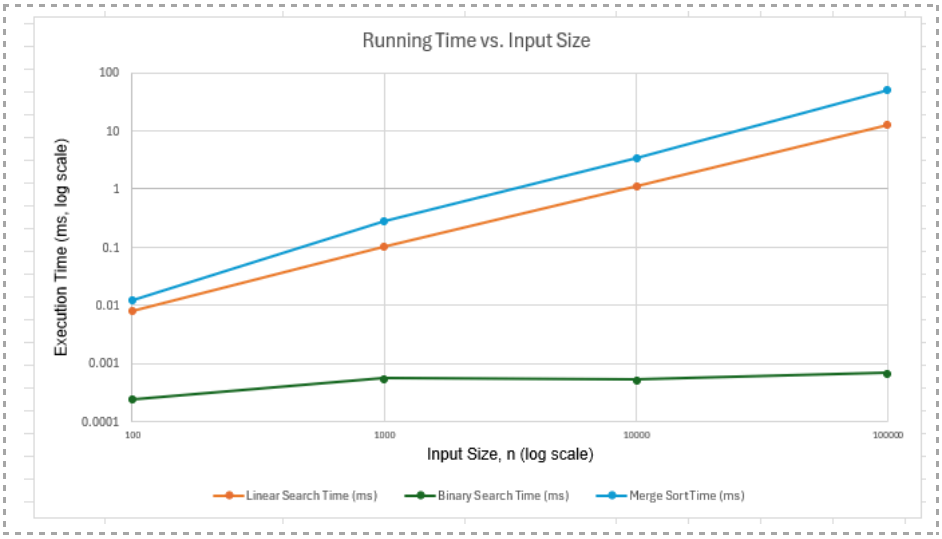


Figure 7. *Average execution time of linear search, binary search, and merge sort for increasing input sizes, plotted on logarithmic axes.*

5.3 Comparison of Theoretical and Experimental Results

In this section we shall compare the expected growth rates from Section 5.1 with the actual growth rates calculated from the experimental results in Section 5.2. To calculate the actual growth rate we take the ratio between the run time for each input size and the run time for the previous input size (e.g. run time for n = 10,000 divided by run time for n = 1,000). Each input size is 10 times larger than the previous so we would expect an O(n) algorithm to take 10 times longer, an O(n log n) algorithm to take slightly longer than 10 times longer and an O(log n) algorithm to take only slightly longer.

Operation	Expected Growth (Big-O)	Observed Growth	Agrees? (Yes / No)
Direct Record Search (Linear Search)	O(n): about 10x per 10x increase in n	About 11-12x per 10x increase in n	Yes
Sorting Student List (Merge Sort)	O(n log n): about 13x per 10x increase in n	About 12-15x per 10x increase in n	Yes
Student Lookup Record (Binary Search)	O(log n): about 1.3x per 10x increase in n	About 1-2x per 10x increase in n	Yes

The results generally match our expectations.For the direct search, the run time increased by about 11-12x as the input size was increased by 10x. This matches our expectation of O(n) complexity, as it should increase by around 10x. For the merge sort, the run time increased by around 12-15x as the input size n was increased by 10x. This is slightly more than expected, but fits the O(n log n) complexity, as we would expect an increase of around 13x. For the first part of the experiment (n = 100 - 1000 we see a higher ratio of around 22x. This is likely due to the run time being so small for low values of n meaning the overhead and noise will have a bigger impact.

Looking at the student lookup, the run time varied very little with n increasing. We saw ratios of around 1-2x as the input size was increased by 10x. This matches our expectations of O(log n) complexity as we would expect an increase of around 1.3x. As the input size is doubled or multiplied, the number of extra steps in the binary search is minimal. However, the run times for the binary search were very small (fractions of a millisecond) so small errors in measurement, caching and background processes can cause fluctuations in the ratio.

Lastly the run times do not match the expected theoretical values exactly due to noise, hardware and constant factors ignored by Big-O notation. However, the results clearly match our expectations. The linear search increased by around 10x, the merge sort increased by slightly more than 10x and the binary search remained almost constant. This shows that the theoretical complexities of O(n), O(n log n) and O(log n) are accurate for each operation.

6. CONCLUSION AND RECOMMENDATIONS

6.1 Conclusion

The Student Attendance Monitoring System has been successfully created to simulate an LMS at educational institutions. The project objectives were met by implementing 5 data structures and algorithms. These included Doubly Linked Lists to store student records, Linked List Queues to check student attendance, Merge Sort to sort students alphabetically, Binary Search to search for students and Linear Search to edit student records. The theoretical time complexity of each algorithm was tested using benchmarking on datasets with up to 100,000 records. The queue had a constant time complexity of O(1), the search algorithms had a quick logarithmic time complexity. This system has replaced the need for manual record keeping, removed any bottlenecks and reduced any clerical errors.

6.2 Recommendations

- **Improve Student Searching** - Using a hash table that is indexed by a unique student ID could provide an average O(1) record lookup, making the searches more efficient as the number of students increases.
- **Strengthen Input Validation** - Validate the attendance dates, status selections, as well as the student IDs before saving. Unique IDs would also distinguish students who might share the EXACT same name.
- **Protect Attendance Records** - Add confirmation prompts before deleting students or changing their already filled-in attendance entries as well as keeping backup copies of saved records so that accident changes can still be recovered.
- **Support of Multiple Classes** - Have the system expanded so that it is able to maintain separate student lists as well as attendance histories for different classes or subjects, allowing teachers to manage several different sections.

7. REFERENCES

Black, P. E. (2021, July 19). *Doubly linked list.* In P. E. Black (Ed.), Dictionary of algorithms and data structures. National Institute of Standards and Technology. Retrieved from: https://xlinux.nist.gov/dads/HTML/doublyLinkedList.html

Black, P. E. (2022, April 21). Binary search. In P. E. Black (Ed.), Dictionary of algorithms and data structures. National Institute of Standards and Technology. Retrieved from: https://xlinux.nist.gov/dads/HTML/binarySearch.html

GeeksforGeeks. (2026a, October 16). *Merge sort*. Retrieved from: https://www.geeksforgeeks.org/dsa/merge-sort/

GeeksforGeeks. (2026b, July 25). *Queue - Linked list implementation*. Retrieved from: https://www.geeksforgeeks.org/dsa/queue-linked-list-implementation/

Free Software Foundation. (2026). *GCC command options. GNU Compiler Collection* (GCC) Documentation. gnu.org. Retrieved from: https://gcc.gnu.org/onlinedocs/gcc/Invoking-GCC.html

Microsoft. (2021a, December 6). <*fstream*>. Microsoft Learn. Retrieved from: https://learn.microsoft.com/en-us/cpp/standard-library/fstream

Microsoft. (2021b, August 3). *Header files (C++)*. Microsoft Learn. Retrieved from: https://learn.microsoft.com/en-us/cpp/cpp/header-files-cpp

Microsoft. (2025, April 23). *steady_clock struct*. Microsoft Learn. Retrieved from: https://learn.microsoft.com/en-us/cpp/standard-library/steady-clock-struct

Programiz. (n.d.). *Linear search*. Retrieved from: https://www.programiz.com/dsa/linear-search

APPENDICES

Appendix A. Source Code

A.1 main.cpp

```
#include "system.h"


// declare shared variables to null

Student* head = NULL;

Student* tail = NULL;

Record* firstRecord = NULL;

Record* lastRecord = NULL;

QueueNode* front = NULL;

QueueNode* rear = NULL;


// for keeping track of next student ID as well as number of students

int nextId = 1;

int studentCount = 0;


// helper function to remove any unneeded spaces, tabs, line endings, from both ends of an
input

string trim(string text) {

    size_t start = text.find_first_not_of(" \t\r\n");

    if (start == string::npos) return "";

    size_t end = text.find_last_not_of(" \t\r\n");

    return text.substr(start, end - start + 1);

}


// helper function to basically convert text to whole number, also rejects extra chars like
letters

bool parseInt(string text, int& value) {

    stringstream input(text);

    char extra;

    if (!(input >> value)) return false;

    return !(input >> extra);

}
```

```
// keep repeating last text until user enters something valid
string getText(string prompt) {
    string text;
    while (true) {
        cout << prompt;
        if (!getline(cin, text)) return "";
        text = trim(text);
        if (!text.empty()) return text;
        cout << "Please enter a value.\n";
    }
}

int getNumber(string prompt, int low, int high) {
    string text;
    int value;
    while (true) {
        cout << prompt;
        if (!getline(cin, text)) return low;
        if (parseInt(text, value) && value >= low && value <= high) return value;
        cout << "Invalid input. Enter " << low << " to " << high << ".\n";
    }
}

string csvText(string text) {
    string result = "\"";
    for (size_t i = 0; i < text.size(); i++) {
        if (text[i] == '"') result += '"';
        result += text[i];
    }
    return result + "\"";
}

bool splitCsv(string line, string fields[], int expected) {
    int count = 0;
    string field;
```

```
    bool quoted = false, closed = false;
    for (size_t i = 0; i < line.size(); i++) {
        char c = line[i];
        if (quoted) {
            if (c == '"') {
                if (i + 1 < line.size() && line[i + 1] == '"') {
                    field += '"';
                    i++;
                } else { quoted = false; closed = true; }
            } else field += c;
        } else if (c == ',') {
            if (count >= expected) return false;
            fields[count++] = field;
            field = "";
            closed = false;
        } else if (c == '"') {
            if (!field.empty() || closed) return false;
            quoted = true;
        } else {
            if (closed) return false;
            field += c;
        }
    }
    if (quoted || count != expected - 1) return false;
    fields[count] = field;
    return true;
}

bool saveData() {
    ofstream file("master.tmp");
    if (!file) { cout << "Cannot open save file. Changes remain in memory.\n"; return false; }
    file << "type,id,name,date,time,status\n";
    Student* student = head;
    while (student != NULL) {
        file << "S," << student->id << ',' << csvText(student->name) << ",,,\n";
```

```
        student = student->next;
    }
    Record* record = firstRecord;
    while (record != NULL) {
        student = findId(record->id);
        file << "A," << record->id << ',' << csvText(student->name) << ','
             << record->date << ',' << record->time << ',' << record->status << '\n';
        record = record->next;
    }
    file.close();
    if (!file) { cout << "Saving failed. Original file was kept.\n"; return false; }
    ifstream old("master.csv");
    bool exists = old.good();
    old.close();
    if (exists) {
        remove("master.bak");
        if (rename("master.csv", "master.bak") != 0) {
            cout << "Cannot replace master.csv. Close it in other programs.\n";
            return false;
        }
    }
    if (rename("master.tmp", "master.csv") != 0) {
        if (exists) rename("master.bak", "master.csv");
        cout << "Saving failed. Check folder permissions.\n";
        return false;
    }
    cout << "Saved to master.csv.\n";
    return true;
}

// read the saved students as well as records and return false if saved format is invalid
bool loadData() {
    ifstream file("master.csv");
    if (!file) {
        // backup failsafe to load from there instead of starting over
```

```
        ifstream backup("master.bak");
        if (backup.good()) {
            cout << "master.csv is missing but master.bak exists. Restore the backup
first.\n";
            return false;
        }
        cout << "Starting a new class.\n";
        return true;
    }
    string line, fields[6];
    if (!getline(file, line) || trim(line) != "type,id,name,date,time,status") return false;
    bool recordsStarted = false;
    while (getline(file, line)) {
        if (!line.empty() && line[line.size() - 1] == '\r') line.erase(line.size() - 1);
        if (!splitCsv(line, fields, 6)) return false;
        int id, status;
        if (!parseInt(fields[1], id) || id < 1 || id > 1000000000) return false;
        // check student IDs & names before adding to linked list
        if (fields[0] == "S") {
            if (recordsStarted || trim(fields[2]).empty() || findId(id) != NULL ||
findName(fields[2]) != NULL) return false;
            if (!fields[3].empty() || !fields[4].empty() || !fields[5].empty()) return false;
            addStudent(id, fields[2]);
        } else if (fields[0] == "A") {
            recordsStarted = true;
            if (findId(id) == NULL || !validDate(fields[3]) || !validTime(fields[4])) return
false;
            if (!parseInt(fields[5], status) || status < 0 || status > 2) return false;
            if (findRecord(id, fields[3], fields[4]) != NULL) return false;
            addRecord(id, fields[3], fields[4], status);
        } else return false;
    }
    return !file.bad();
}

// delete dynamically allocated queue as well as list nodes before completion
```

```
void clearData() {
    while (!isEmpty()) dequeue();
    while (firstRecord != NULL) removeRecord(firstRecord);
    while (head != NULL) {
        Student* next = head->next;
        delete head;
        head = next;
    }
    tail = NULL;
    studentCount = 0;
    nextId = 1;
}


// entry point (argc & argv for reading command line options)
int main(int argc, char* argv[]) {
    // seed random num gen for random grouping later
    srand((unsigned int)time(NULL));
    // run timing tests if --benchmark is here
    if (argc > 1 && string(argv[1]) == "--benchmark") {
        runBenchmark();
        return 0;
    }
    cout << "STUDENT ATTENDANCE MONITORING SYSTEM\n";
    // stop if loading fails; invalid save not overwrriten
    if (!loadData()) {
        cout << "Cannot load saved data safely. Check master.csv; it has not been
overwritten.\n";
        clearData();
        return 1;
    }


    // show menu again after each action while input available
    while (cin) {
        cout << "\n1. Check attendance\n2. Manage students\n3. View / correct attendance
history\n4. Search student\n5. Generate groups\n6. Save\n0. Save and exit\n";
        int choice = getNumber("Choice: ", 0, 6);
```

```
        if (!cin || choice == 0) {
            if (saveData() || !cin) break;
            cout << "Save failed. You can retry or choose to exit without saving.\n";
            if (getNumber("1 = Exit without saving, 0 = Return: ", 0, 1) == 1) break;
        } else if (choice == 1) checkAttendance();
        else if (choice == 2) manageStudents();
        else if (choice == 3) historyMenu();
        else if (choice == 4) searchMenu();
        else if (choice == 5) groupStudents();
        else saveData();
    }
    clearData();
    cout << "Program ended.\n";
    return 0;
}
```

A.2 students.cpp

```
#include "system.h"

Student* findId(int id) {
    Student* current = head;
    while (current != NULL) {
        if (current->id == id) return current;
        current = current->next;
    }
    return NULL;
}

Student* findName(string name) {
    Student* current = head;
    while (current != NULL) {
        if (nameKey(current->name) == nameKey(name)) return current;
        current = current->next;
    }
    return NULL;
```

```
}


void addStudent(int id, string name) {
    Student* item = new Student;
    item->id = id;
    item->name = name;
    item->prev = tail;
    item->next = NULL;
    if (tail == NULL) head = item;
    else tail->next = item;
    tail = item;
    studentCount++;
    if (id >= nextId) nextId = id + 1;
}


void showStudents() {
    if (studentCount == 0) { cout << "No students yet.\n"; return; }
    Student** list = sortedStudents();
    cout << "\nID       NAME\n";
    for (int i = 0; i < studentCount; i++) cout << left << setw(9) << list[i]->id <<
list[i]->name << '\n';
    delete[] list;
}


void manageStudents() {
    cout << "\n1. Add student\n2. Display students\n3. Edit name\n4. Delete student\n5. Import
names\n0. Back\n";
    int choice = getNumber("Choice: ", 0, 5);
    if (!cin || choice == 0) return;
    if (choice == 2) { showStudents(); return; }
    if (choice == 1) {
        string name = getText("Full name (Surname, First Name): ");
        if (!cin) return;
        if (findName(name) != NULL) { cout << "That name already exists.\n"; return; }
        addStudent(nextId, name);
    } else if (choice == 5) {
```

```
        string path = getText("Text file path (one full name per line): ");
        if (!cin) return;
        ifstream file(path.c_str());
        if (!file) { cout << "File not found.\n"; return; }
        string name;
        int added = 0, skipped = 0;
        while (getline(file, name)) {
            name = trim(name);
            if (name.empty() || findName(name) != NULL) { skipped++; continue; }
            addStudent(nextId, name);
            added++;
        }
        cout << "Added: " << added << " | Skipped blank/duplicate names: " << skipped << '\n';
    } else {
        showStudents();
        if (studentCount == 0) return;
        int id = getNumber("Student ID (0 to cancel): ", 0, 1000000000);
        if (!cin || id == 0) return;
        Student* student = findId(id);
        if (student == NULL) { cout << "Student not found.\n"; return; }
        if (choice == 3) {
            string name = getText("New full name: ");
            if (!cin) return;
            Student* duplicate = findName(name);
            if (duplicate != NULL && duplicate != student) { cout << "That name already
exists.\n"; return; }
            student->name = name;
        } else {
            cout << "Delete " << student->name << " and ALL their attendance history?\n";
            if (getNumber("1 = Yes, 0 = Cancel: ", 0, 1) != 1 || !cin) return;
            Record* current = firstRecord;
            while (current != NULL) {
                Record* next = current->next;
                if (current->id == id) removeRecord(current);
                current = next;
```

```
            }
            if (student->prev != NULL) student->prev->next = student->next;
            else head = student->next;
            if (student->next != NULL) student->next->prev = student->prev;
            else tail = student->prev;
            delete student;
            studentCount--;
        }
    }
    saveData();
}
```

A.3 attendance.cpp

```
#include "system.h"

// This checks if the date follows the right format and if it is a valid date in the calendar.
bool validDate(string date){
    if(date.size() != 10 || date[4] != '-' || date[7] != '-'){
        return false;
    }
    for(int i = 0; i < 10; i++){
        if (i != 4 && i != 7 && (date[i] < '0' || date[i] > '9')){
            return false;
        }
    }
    int year = atoi(date.substr(0, 4).c_str());
    int month = atoi(date.substr(5, 2).c_str());
    int day = atoi(date.substr(8, 2).c_str());
    int days[12] = {31,28,31,30,31,30,31,31,30,31,30,31};

    if(year < 1900 || month < 1 || month > 12){
    return false;
    }
    if(year % 400 == 0 || (year % 4 == 0 && year % 100 != 0)){
        days[1] = 29;
```

```
    }
    return day >= 1 && day <= days[month - 1];
}

bool validTime(string time){
    if(time.size() != 5 || time[2] != ':'){
        return false;
    }
    for(int i = 0; i < 5; i++){
        if(i != 2 && (time[i] < '0' || time[i] > '9')){
            return false;
        }
    }
    int hour = atoi(time.substr(0, 2).c_str());
    int minute = atoi(time.substr(3, 2).c_str());
    return hour < 24 && minute < 60;
}

void enqueue(Student* student){
    QueueNode* item = new QueueNode;
    item->student = student;
    item->next = NULL;
    if(rear == NULL){
        front = item;
    }
    else{
        rear->next = item;
    }
    rear = item;
}

bool isEmpty(){
    return front == NULL;
}
```

```
Student* peek(){
    if(isEmpty()){
        return NULL;
    }
    return front->student;
}

void dequeue(){
    if(isEmpty()){
        return;
    }
    QueueNode* item = front;
    front = front->next;
    if(front == NULL){
        rear = NULL;
    }
    delete item;
}

void checkAttendance(){
    if(studentCount == 0){
        cout << "Add students first.\n";
        return;
    }
    string date, time;
    do{
        date = getText("Session date (YYYY-MM-DD): ");
        if(!cin){
            return;
        }
        if(!validDate(date)){
            cout << "Invalid date.\n";
        }
    } while (!validDate(date));
    do{
```

```
        time = getText("Session time (HH:MM, 24-hour): ");
        if(!cin){
            return;
        }
        if(!validTime(time)){
            cout << "Invalid time.\n";
        }
    }while(!validTime(time));
    Student** list = sortedStudents();
    int skipped = 0;

    //Only add the student's that do not have attendance record for just this session in time
    for(int i = 0; i < studentCount; i++){
        if(findRecord(list[i]->id, date, time) == NULL){
            enqueue(list[i]);
        }
        else{
            skipped++;
        }
    }
    delete[] list;
    cout << "Existing entries skipped: " << skipped
         << "\n2 = Present | 1 = Late | 0 = Absent | 3 = Stop\n";
    while(!isEmpty()){
        Student* student = peek();
        cout << "\n" << student->id << " - " << student->name << '\n';
        int status = getNumber("Status: ", 0, 3);
        if(!cin || status == 3){
            break;
        }
        addRecord(student->id, date, time, status);
        dequeue();
    }
    while(!isEmpty()){
        dequeue();
```

```
    }
    saveData();
    cout << "Unmarked students remain unrecorded. Repeat this session to continue.\n";
}
```

A.4 history.cpp

```
#include "system.h"

void addRecord(int id, string date, string time, int status) {
    Record* item = new Record;
    item->id = id;
    item->date = date;
    item->time = time;
    item->status = status;
    item->prev = lastRecord;
    item->next = NULL;
    if (lastRecord == NULL) firstRecord = item;
    else lastRecord->next = item;
    lastRecord = item;
}

Record* findRecord(int id, string date, string time) {
    Record* current = firstRecord;
    while (current != NULL) {
        if (current->id == id && current->date == date && current->time == time) return
current;
        current = current->next;
    }
    return NULL;
}

string statusText(int status) {
    if (status == 2) return "Present";
```

```
    if (status == 1) return "Late";
    return "Absent";
}

void removeRecord(Record* item) {
    if (item->prev != NULL) item->prev->next = item->next;
    else firstRecord = item->next;
    if (item->next != NULL) item->next->prev = item->prev;
    else lastRecord = item->prev;
    delete item;
}

void showHistory(int id) {
    int counts[3] = {0,0,0};
    Record* current = firstRecord;
    cout << "\nDATE        TIME    STATUS\n";
    while (current != NULL) {
        if (current->id == id) {
            cout << current->date << "  " << current->time << "   " <<
statusText(current->status) << '\n';
            counts[current->status]++;
        }
        current = current->next;
    }
    cout << "Present: " << counts[2] << " | Late: " << counts[1] << " | Absent: " << counts[0]
<< '\n';
    if (counts[0] + counts[1] + counts[2] == 0) cout << "No attendance records.\n";
}

void historyMenu() {
    showStudents();
    if (studentCount == 0) return;
    int id = getNumber("Student ID (0 to cancel): ", 0, 1000000000);
    if (!cin || id == 0) return;
    Student* student = findId(id);
    if (student == NULL) { cout << "Student not found.\n"; return; }
```

```
    cout << student->name << '\n';
    showHistory(id);
    if (getNumber("1 = Correct an entry, 0 = Back: ", 0, 1) == 0 || !cin) return;
    string date = getText("Entry date (YYYY-MM-DD): ");
    if (!cin) return;
    string time = getText("Entry time (HH:MM): ");
    if (!cin) return;
    Record* record = findRecord(id, date, time);
    if (record == NULL) { cout << "Attendance entry not found.\n"; return; }
    int status = getNumber("New status (2 = Present, 1 = Late, 0 = Absent): ", 0, 2);
    if (!cin) return;
    record->status = status;
    saveData();
}
```

A.5 search.cpp

```
#include "system.h"


string nameKey(string text) {
    for (size_t i = 0; i < text.size(); i++) {
        if (text[i] >= 'A' && text[i] <= 'Z') text[i] += 'a' - 'A';
    }
    return text;
}


void merge(Student* list[], Student* temp[], int left, int middle, int right) {
    int i = left, j = middle + 1, k = left;
    while (i <= middle && j <= right) {
        if (nameKey(list[i]->name) <= nameKey(list[j]->name)) temp[k++] = list[i++];
        else temp[k++] = list[j++];
    }
    while (i <= middle) temp[k++] = list[i++];
    while (j <= right) temp[k++] = list[j++];
    for (int p = left; p <= right; p++) list[p] = temp[p];
}
```

```
void mergeSort(Student* list[], Student* temp[], int left, int right) {
    if (left >= right) return;
    int middle = left + (right - left) / 2;
    mergeSort(list, temp, left, middle);
    mergeSort(list, temp, middle + 1, right);
    merge(list, temp, left, middle, right);
}

Student** sortedStudents() {
    Student** list = new Student*[studentCount];
    Student** temp = new Student*[studentCount];
    Student* current = head;
    for (int i = 0; i < studentCount; i++) {
        list[i] = current;
        current = current->next;
    }
    mergeSort(list, temp, 0, studentCount - 1);
    delete[] temp;
    return list;
}

int binarySearch(Student* list[], int count, string name) {
    int low = 0, high = count - 1;
    string target = nameKey(name);
    while (low <= high) {
        int middle = low + (high - low) / 2;
        string key = nameKey(list[middle]->name);
        if (key == target) return middle;
        if (key < target) low = middle + 1;
        else high = middle - 1;
    }
    return -1;
}
```

```
void searchMenu() {
    if (studentCount == 0) { cout << "No students yet.\n"; return; }
    string name = getText("Exact full name to search (case-insensitive): ");
    if (!cin) return;
    Student** list = sortedStudents();
    int index = binarySearch(list, studentCount, name);
    if (index == -1) cout << "Student not found.\n";
    else {
        cout << "Found: " << list[index]->id << " - " << list[index]->name << '\n';
        showHistory(list[index]->id);
    }
    delete[] list;
}


void runBenchmark() {
    int sizes[4] = {100, 1000, 10000, 100000};
    ofstream output("benchmark_results.csv");
    if (!output) { cout << "Cannot create benchmark output.\n"; return; }
    output <<
"n,linear_search_ms,binary_search_ms,merge_sort_ms,repetitions,searches_per_run\n";
    volatile int found = 0;
    for (int s = 0; s < 4; s++) {
        int n = sizes[s];
        for (int i = 1; i <= n; i++) {
            stringstream name;
            name << "Student " << setfill('0') << setw(6) << i;
            addStudent(i, name.str());
        }
        Student** list = sortedStudents();
        Student** temp = new Student*[n];
        double linearTotal = 0, binaryTotal = 0, sortTotal = 0;
        string target = "ZZZ Missing Student";
        for (int run = 0; run < 5; run++) {
            chrono::steady_clock::time_point start = chrono::steady_clock::now();
            for (int k = 0; k < 100; k++) found = (findName(target) != NULL);
```

```
            chrono::steady_clock::time_point end = chrono::steady_clock::now();
            linearTotal += chrono::duration<double, milli>(end - start).count() / 100;
            start = chrono::steady_clock::now();
            for (int k = 0; k < 100; k++) found = binarySearch(list, n, target);
            end = chrono::steady_clock::now();
            binaryTotal += chrono::duration<double, milli>(end - start).count() / 100;
            for (int i = 0; i < n / 2; i++) {
                Student* item = list[i];
                list[i] = list[n - 1 - i];
                list[n - 1 - i] = item;
            }
            start = chrono::steady_clock::now();
            mergeSort(list, temp, 0, n - 1);
            end = chrono::steady_clock::now();
            sortTotal += chrono::duration<double, milli>(end - start).count();
        }
        output << n << ',' << fixed << setprecision(9) << linearTotal / 5 << ',' <<
binaryTotal / 5 << ',' << sortTotal / 5 << ",5,100\n";
        cout << "Finished n = " << n << '\n';
        delete[] temp;
        delete[] list;
        clearData();
    }
    output.close();
    if (!output) cout << "Benchmark output could not be saved.\n";
    else cout << "Saved benchmark_results.csv. Check value: " << found << '\n';
}
```

A.6 grouping.cpp

```
#include "system.h"


void groupStudents() {
    // Checks if students are available
    if (studentCount == 0) { cout << "Add students first.\n"; return; }
```

```
    // Get the number of groups
    int groups = getNumber("Number of groups: ", 1, studentCount);
    if (!cin) return;

    // Select grouping mode
    int mode = getNumber("1 = Alphabetical round-robin, 2 = Random: ", 1, 2);
    if (!cin) return;

    // Get sorted student list
    Student** list = sortedStudents();
    if (mode == 2) {
        for (int i = studentCount - 1; i > 0; i--) {
            int j = rand() % (i + 1);
            Student* temp = list[i];
            list[i] = list[j];
            list[j] = temp;
        }
    }

    // Display students in each group
    for (int group = 0; group < groups; group++) {
        cout << "\nGROUP " << group + 1 << '\n';

        // Assign students using round-robin
        for (int i = group; i < studentCount; i += groups)
            cout << list[i]->id << " - " << list[i]->name << '\n';
    }

    // Free allocated memory
    delete[] list;
}
```

Appendix B. Team Composition

#	Name (Surname, First Name M.I.)	Student No.	Role	Main Contributions	%
1	Alvarez, Joshua Caleb P	2515614	Documentation/ students records developer	'students.cpp'	100
2	Baltazar, Kenji	2515531	Figure and Flowchart Creators/ Code Submaster/ Powerpoint	'attendance.cpp'	100
3	Formilleza, Huone Anthony	2510807	Record Systems Programmer / History system	'history.cpp'	100
4	Garcia, Vince Joseph R.	2515512	Documentator / Searching developer	'search.cpp'	100
5	Medina, Jomz Ryle E.	2514659	Project Leader/ Code Master/ Code Presenter	'main.cpp', 'system.h'	100
6	Regala, Joaquin Luiz B.	2515588	Documentation/ Grouping system	'grouping.cpp'	100

Appendix C. GitHub Repository

Item	Details
Repository link	https://github.com/Medinacpe/Group-1-CPE21S1-Attendance-Monitoring-Final-Project
Branch / release submitted	main
README contents	Project description, system features, data structures and algorithms used, source file descriptions, compilation and execution instructions, sample input, and team members.
How to compile	g++ -std=c++11 *.cpp -o attendance.exe

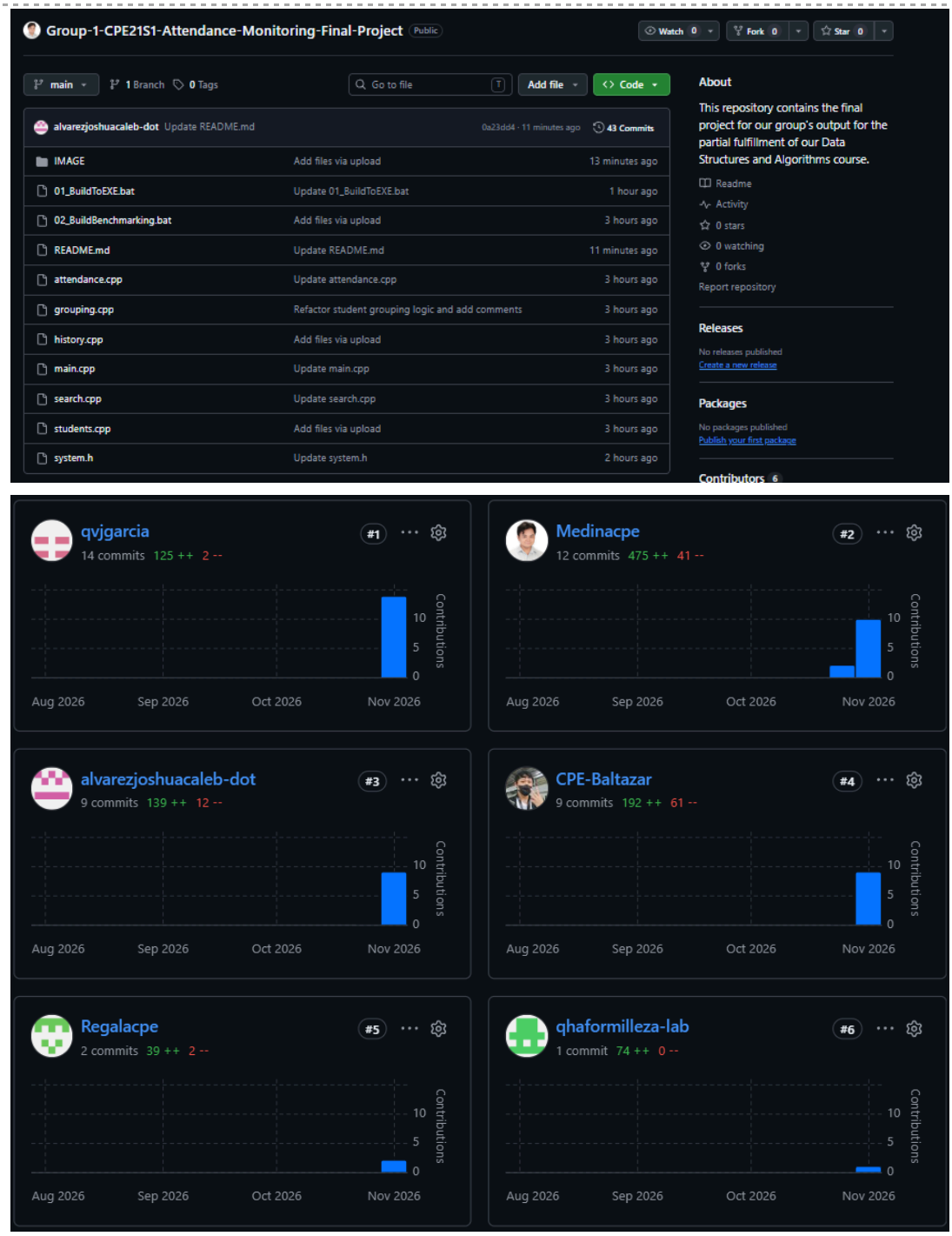


Figure 8. Screenshot of the repository page showing files and commit history

EVALUATION CRITERIA

Criteria	Description	Points
Business Process	Realistic process clearly described; problems and proposed system well defined	15
Data Structures and Algorithms Applied	At least five course topics, correctly explained and justified for the business process	25
System Flowchart	Complete, correct symbols, matches the program	10
Output	All features shown working, including invalid input handling	15
Performance Analysis	Correct theoretical complexity; sound experimental method; meaningful comparison	25
Documentation and Appendices	Complete code, team composition, working GitHub repository; clear writing	10
Total		100